

Q1. What is the Spring Framework?

Answer

Spring is a lightweight Java framework used to build enterprise applications. It provides features like Dependency Injection (DI), Inversion of Control (IoC), transaction management, AOP, and easy integration with databases and web applications.

Simple Example

Instead of creating objects manually using `new`, Spring creates and manages them.

Real Project Usage

In almost every Spring Boot application, Spring manages services, repositories, and controllers automatically.

Interviewer Cross Question

Why is Spring called a lightweight framework?

Q2. Why do we use Spring?

Answer

Spring reduces boilerplate code, promotes loose coupling, simplifies object creation, manages dependencies automatically, and makes applications easier to test and maintain.

Simple Example

Without Spring:

```
EmployeeService service = new EmployeeService();
```

With Spring:

```
@Autowired  
EmployeeService service;
```

Real Project Usage

Large enterprise projects use Spring because components remain independent and easier to maintain.

Q3. What is IoC (Inversion of Control)?

Answer

IoC means the control of creating and managing objects is transferred from the programmer to the Spring container.

Simple Example

Instead of writing:

```
Car car = new Car();
```

Spring creates the `Car` object and provides it whenever required.

Real Project Usage

Controllers, Services, and Repositories are all created by Spring.

Interviewer Cross Question

Who controls object creation in Spring?

Answer: **Spring Container.**

Q4. What is Dependency Injection (DI)?

Answer

Dependency Injection is a design pattern where Spring injects the required object into another object instead of the object creating it itself.

Simple Example

```
class Car{
    Engine engine;
}
```

Spring injects the `Engine` object into `Car`.

Real Project Usage

A Service receives a Repository object automatically through constructor injection.

Q5. What is the difference between IoC and DI?

Answer

- IoC is the overall concept of giving object management to Spring.
- DI is one technique used by Spring to achieve IoC.

Simple Example

IoC = Company hires employees.

DI = HR assigns the employee to a project.

Real Project Usage

Every Spring application uses IoC, while DI is used to inject required beans.

Q6. What is a Spring Bean?

Answer

A Spring Bean is an object that is created, configured, and managed by the Spring Container.

Simple Example

```
@Service
public class EmployeeService{}
```

EmployeeService becomes a Spring Bean.

Real Project Usage

Controllers, Services, Repositories, Components, and Configurations are all Spring Beans.

Q7. What is the Spring Container?

Answer

The Spring Container is responsible for creating, configuring, managing, and destroying Spring Beans.

Simple Example

When the application starts, the container scans all Spring components and creates their objects.

Real Project Usage

Every Spring Boot application starts an ApplicationContext, which acts as the Spring Container.

Q8. What is the difference between BeanFactory and ApplicationContext?

Answer

BeanFactory	ApplicationContext
Basic container	Advanced container
Lazy loading	Eager loading by default
Limited features	Supports events, AOP, i18n

Simple Example

Almost every Spring Boot project uses **ApplicationContext**.

Real Project Usage

ApplicationContext is the standard container in modern Spring applications.

Interviewer Cross Question

Which container is commonly used in Spring Boot?

Answer: **ApplicationContext**.

Q9. What is Loose Coupling?

Answer

Loose coupling means one class depends on an interface instead of a specific implementation, making the application flexible and maintainable.

Simple Example

Instead of

```
Car car = new BMW();
```

Use

```
Car car = vehicle;
```

Spring decides which implementation to inject.

Real Project Usage

Changing from MySQL to PostgreSQL usually requires minimal code changes because of loose coupling.

Q10. What is Tight Coupling?

Answer

Tight coupling means one class directly depends on another concrete class, making changes difficult.

Simple Example

```
EmployeeService service = new EmployeeService();
```

The object is created manually, so changing the implementation requires code changes.

Real Project Usage

Modern Spring applications avoid tight coupling by using Dependency Injection.

Q11. What is @Component?

Answer

@Component is a generic Spring annotation used to mark a class as a Spring Bean. During component scanning, Spring automatically detects it and creates an object.

Simple Example

```
@Component
public class EmailUtil {
}
```

Real Project Usage

Used for utility classes, helper classes, validators, and custom components.

Interviewer Cross Question

Can Spring create a Bean without XML?

Answer: Yes, using annotations like @Component.

Q12. What is @Service?

Answer

@Service is a specialization of @Component used for business logic classes. It makes the code more readable by indicating that the class belongs to the service layer.

Simple Example

```
@Service
public class EmployeeService {
}
```

Real Project Usage

Handles business operations such as salary calculation, order processing, and user management.

Interviewer Cross Question

Does @Service create a Bean?

Answer: Yes. It behaves like @Component.

Q13. What is @Repository?

Answer

@Repository marks a class as the Data Access Object (DAO) layer. It communicates with the database and provides automatic exception translation for persistence exceptions.

Simple Example

```
@Repository
public class EmployeeRepository {
}
```

Real Project Usage

Used with Spring Data JPA, Hibernate, and JDBC to perform database operations.

Interviewer Cross Question

Which layer communicates with the database?

Answer: Repository layer.

Q14. What is @Controller?

Answer

@Controller is used in Spring MVC to handle HTTP requests and return a view such as a JSP or Thymeleaf page.

Simple Example

```
@Controller
public class HomeController {
}
```

Real Project Usage

Used in web applications that return HTML pages.

Interviewer Cross Question

Does @Controller return JSON by default?

Answer: No. It usually returns a view.

Q15. What is @RestController?

Answer

@RestController is a combination of @Controller and @ResponseBody. It returns data such as JSON or XML directly to the client instead of rendering a view.

Simple Example

```
@RestController
public class EmployeeController {
}
```

Real Project Usage

Used in REST APIs consumed by React, Angular, mobile apps, and other services.

Interviewer Cross Question

What is the difference between @Controller and @RestController?

Answer:

- `@Controller` → Returns a View.
- `@RestController` → Returns JSON/XML.

Q16. What is `@Autowired`?

Answer

`@Autowired` tells Spring to automatically inject the required Bean into another Bean. Spring resolves the dependency from the application context.

Simple Example

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;
}
```

Real Project Usage

Used to inject Services, Repositories, Utility classes, and Configuration Beans.

Interviewer Cross Question

Can `@Autowired` be used on constructors?

Answer: Yes, and constructor injection is the recommended approach.

Q17. What is Constructor Injection?

Answer

Constructor Injection is a dependency injection technique where required dependencies are provided through the class constructor. It is the recommended method because it promotes immutability and makes dependencies mandatory.

Simple Example

```
@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
```



```
        this.repository = repository;
    }
}
```

Real Project Usage

Most modern Spring Boot applications use constructor injection for all required dependencies.

Interviewer Cross Question

Why is constructor injection preferred?

Answer: It makes dependencies mandatory, improves testability, and avoids `NullPointerException` caused by missing dependencies.

Q18. What is Setter Injection?

Answer

Setter Injection injects dependencies through setter methods. It is useful for optional dependencies but is less preferred than constructor injection.

Simple Example

```
@Service
public class EmployeeService {

    private EmployeeRepository repository;

    @Autowired
    public void setRepository(EmployeeRepository repository)
    {
        this.repository = repository;
    }
}
```

Real Project Usage

Used when a dependency is optional or may be changed after object creation.

Interviewer Cross Question

Which is better: Constructor Injection or Setter Injection?

Answer: Constructor Injection is generally preferred.

Q19. What is @Qualifier?

Answer

`@Qualifier` is used when multiple Beans of the same type exist. It tells Spring exactly which Bean should be injected.

Simple Example

```
@Autowired
@Qualifier("paypalPayment")
private PaymentService paymentService;
```

Real Project Usage

Useful when an application has multiple implementations, such as PayPal, Stripe, and Razorpay payment services.

Interviewer Cross Question

Why do we use `@Qualifier`?

Answer: To resolve ambiguity when multiple Beans of the same type are available.

Q20. What is `@Primary`?

Answer

`@Primary` marks one Bean as the default choice when multiple Beans of the same type exist and no `@Qualifier` is specified.

Simple Example

```
@Primary
@Component
public class StripePaymentService implements PaymentService {
}
```

Real Project Usage

One payment provider or one cache implementation can be set as the default while allowing others to coexist.

Interviewer Cross Question

What happens if both `@Primary` and `@Qualifier` are used?

Answer: `@Qualifier` takes precedence because it explicitly specifies which Bean to inject.

Next 10 Questions (21–30)

The next set will cover more Spring Core concepts commonly asked in interviews:

21. Bean Scope (Overview)
22. Singleton Scope
23. Prototype Scope
24. Request Scope
25. Session Scope
26. Bean Lifecycle
27. `@PostConstruct`
28. `@PreDestroy`
29. Component Scanning (`@ComponentScan`)
30. Spring Configuration (`@Configuration` and `@Bean`)

These questions are frequently asked in **0–2 years** Java/Spring interviews and naturally follow the dependency injection topics.

next 10

Spring Core Interview Questions (21–30)

Q21. What are Bean Scopes in Spring?

Answer

Bean Scope defines how many instances of a Spring Bean are created and how long they live. Spring provides several scopes such as Singleton, Prototype, Request, Session, Application, and WebSocket.

Simple Example

One Bean can be shared across the entire application or a new object can be created every time it is requested.

Real Project Usage

- Singleton → Services, Repositories
- Prototype → Report generators, temporary objects
- Request → REST request data
- Session → Logged-in user information

Interviewer Cross Question

What is the default Bean scope in Spring?

Answer: Singleton.

Q22. What is Singleton Scope?

Answer

Singleton is the default Bean scope in Spring. Only one object of the Bean is created for the entire Spring Container, and all users share the same instance.

Simple Example

```
@Service
public class EmployeeService {
}
```

Only one `EmployeeService` object exists.

Real Project Usage

Used for Services, Repositories, Configuration classes, and Utility Beans.

Interviewer Cross Question

Is Singleton scope thread-safe?

Answer: Not automatically. Developers should avoid storing mutable user-specific data in singleton beans.

Q23. What is Prototype Scope?

Answer

Prototype scope creates a new Bean object every time it is requested from the Spring Container.

Simple Example

```
@Component
@Scope("prototype")
public class ReportGenerator {
}
```

Every request gets a new `ReportGenerator` object.

Real Project Usage

Used for temporary objects such as PDF generators, invoice builders, and report creators.

Interviewer Cross Question

How many objects are created in Prototype scope?

Answer: A new object is created every time the Bean is requested.

Q24. What is Request Scope?

Answer

Request scope creates one Bean for each HTTP request. The Bean exists only during that request and is destroyed afterward.

Simple Example

```
@Component
@RequestScope
public class UserRequest {
}
```

Each HTTP request gets its own `UserRequest` object.

Real Project Usage

Used to store request-specific information such as request IDs, request headers, or temporary request data.

Interviewer Cross Question

Can Request scope be used in a normal Java application?

Answer: No. It is available only in web applications.

Q25. What is Session Scope?

Answer

Session scope creates one Bean for each user session. The Bean remains available until the user logs out or the session expires.

Simple Example

```
@Component
@SessionScope
public class ShoppingCart {
}
```

Each logged-in user gets a separate `ShoppingCart`.

Real Project Usage

Used for shopping carts, user preferences, and authenticated user information.

Interviewer Cross Question

Is Session scope shared between users?

Answer: No. Every user has a separate session Bean.

Q26. What is the Bean Lifecycle?

Answer

The Bean Lifecycle describes the stages a Spring Bean goes through: creation, dependency injection, initialization, usage, and destruction.

Simple Example

```
Bean Created
    ↓
Dependencies Injected
    ↓
@PostConstruct
    ↓
Bean Ready
    ↓
Application Uses Bean
    ↓
@PreDestroy
```

Real Project Usage

Database connections, cache initialization, and resource cleanup are often performed during the Bean lifecycle.

Interviewer Cross Question

Which annotation runs after dependency injection?

Answer: @PostConstruct.

Q27. What is @PostConstruct?

Answer

`@PostConstruct` is executed once after the Bean is created and all dependencies have been injected. It is used for initialization tasks.

Simple Example

```
@PostConstruct
public void init() {
    System.out.println("Bean Initialized");
}
```

Real Project Usage

Loading configuration, initializing caches, or opening resources after the application starts.

Interviewer Cross Question

How many times does `@PostConstruct` execute for a Singleton Bean?

Answer: Only once.

Q28. What is `@PreDestroy`?

Answer

`@PreDestroy` is executed just before the Bean is destroyed. It is used to release resources such as database connections or file handles.

Simple Example

```
@PreDestroy
public void cleanup() {
    System.out.println("Cleaning resources");
}
```

Real Project Usage

Closing database connections, shutting down thread pools, and releasing external resources.

Interviewer Cross Question

Will `@PreDestroy` be called for Prototype Beans?

Answer: Generally, no. Spring manages the creation of prototype beans but does not automatically manage their full lifecycle after creation.

Q29. What is @ComponentScan?

Answer

@ComponentScan tells Spring where to search for classes annotated with @Component, @Service, @Repository, and @Controller.

Simple Example

```
@Configuration
@ComponentScan("com.company")
public class AppConfig {
}
```

Spring scans the `com.company` package and registers all eligible Beans.

Real Project Usage

Used in almost every Spring application to automatically discover application components.

Interviewer Cross Question

What happens if @ComponentScan is not configured correctly?

Answer: Spring may not find your Beans, leading to errors such as "No qualifying bean found."

Q30. What are @Configuration and @Bean?

Answer

@Configuration marks a class as a configuration class, while @Bean defines a Bean that Spring should manage. This approach is called Java-based configuration.

Simple Example

```
@Configuration
public class AppConfig {

    @Bean
    public EmployeeService employeeService() {
        return new EmployeeService();
    }
}
```

Real Project Usage

Used to configure third-party libraries, database connections, security settings, and custom Beans that cannot be annotated directly.

Interviewer Cross Question

When should you use @Bean instead of @Component?

Answer: Use @Bean when you need to create or configure objects from third-party libraries or when you want explicit control over Bean creation.

Q31. What is Spring Boot?

Answer

Spring Boot is an extension of the Spring Framework that makes it easy to create stand-alone, production-ready applications. It reduces configuration, provides embedded servers, and simplifies dependency management.

Simple Example

Instead of configuring XML files and Tomcat manually, Spring Boot lets you start an application with just a few annotations.

Real Project Usage

Most modern enterprise applications are built using Spring Boot because development is faster and deployment is simpler.

Interviewer Cross Question

Is Spring Boot a replacement for Spring?

Answer: No. Spring Boot is built on top of the Spring Framework and uses all Spring features.

Q32. What is the difference between Spring and Spring Boot?

Answer

Spring is a framework that requires more configuration, while Spring Boot simplifies development by providing auto-configuration, starter dependencies, and embedded servers.

Simple Example

Spring	Spring Boot
More configuration	Minimal configuration
External Tomcat required	Embedded Tomcat
Manual dependency setup	Starter dependencies
More setup time	Faster development

Real Project Usage

Older projects may use Spring Framework, while most new applications use Spring Boot.

Interviewer Cross Question

Can we use Spring without Spring Boot?

Answer: Yes, but it requires more manual configuration.

Q33. What are the advantages of Spring Boot?

Answer

Spring Boot offers:

- Auto Configuration
- Starter Dependencies
- Embedded Server
- Faster development
- Production-ready features like health checks and metrics

Simple Example

A REST API can be created with very little configuration.

Real Project Usage

Teams use Spring Boot to reduce development time and standardize application setup.

Interviewer Cross Question

Why do companies prefer Spring Boot?

Answer: Because it reduces boilerplate code and speeds up development.

Q34. What is Auto Configuration?

Answer

Auto Configuration automatically configures Spring Beans based on the libraries available in the project and the application configuration.

Simple Example

If `spring-boot-starter-data-jpa` is added, Spring Boot automatically configures JPA components.

Real Project Usage

Developers don't need to manually configure `DataSource`, `DispatcherServlet`, or `Jackson` in most cases.

Interviewer Cross Question

Can Auto Configuration be customized?

Answer: Yes. You can override the default configuration by defining your own Beans or changing properties.

Q35. What are Starter Dependencies?

Answer

Starter Dependencies are predefined dependency bundles that include all the required libraries for a specific feature.

Simple Example

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

This includes Spring MVC, Embedded Tomcat, Jackson, and validation libraries.

Real Project Usage

Instead of adding many individual dependencies, developers add one starter dependency.

Interviewer Cross Question

Name some commonly used starter dependencies.

Answer:

- spring-boot-starter-web
- spring-boot-starter-data-jpa
- spring-boot-starter-security
- spring-boot-starter-test

Q36. What is @SpringBootApplication?

Answer

@SpringBootApplication is the main annotation used to start a Spring Boot application. It combines three annotations:

- @Configuration
- @EnableAutoConfiguration
- @ComponentScan

Simple Example

```
@SpringBootApplication
public class EmployeeApplication {

    public static void main(String[] args) {
        SpringApplication.run(EmployeeApplication.class,
args);
    }
}
```

Real Project Usage

Every Spring Boot application starts with a class annotated using @SpringBootApplication.

Interviewer Cross Question

Can we replace @SpringBootApplication with individual annotations?

Answer: Yes. It is simply a combination of three Spring annotations.

Q37. What is Embedded Tomcat?

Answer

Embedded Tomcat is a web server packaged inside a Spring Boot application. It allows the application to run without installing an external Tomcat server.

Simple Example

Run:

```
mvn spring-boot:run
```

The application starts with its own Tomcat server.

Real Project Usage

Most Spring Boot REST APIs are deployed as executable JAR files with Embedded Tomcat.

Interviewer Cross Question

Can Spring Boot use servers other than Tomcat?

Answer: Yes. It also supports Jetty and Undertow.

Q38. What is `application.properties`?

Answer

`application.properties` is the default configuration file used to store application settings such as server port, database details, logging, and custom properties.

Simple Example

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/companydb
```

Real Project Usage

Database configuration, API keys, logging levels, and environment-specific settings are stored in this file.

Interviewer Cross Question

Can Spring Boot use YAML instead of properties?

Answer: Yes. It supports both `application.properties` and `application.yml`.

Q39. What are Spring Profiles?

Answer

Spring Profiles allow different configurations for different environments such as Development, Testing, and Production.

Simple Example

```
spring.profiles.active=dev
```

Separate files:

- application-dev.properties
- application-test.properties
- application-prod.properties

Real Project Usage

Different database URLs and logging levels are used for development and production environments.

Interviewer Cross Question

Why are Profiles useful?

Answer: They allow the same application to use different configurations without changing the code.

Q40. How does a Spring Boot application start?

Answer

A Spring Boot application starts when the `main()` method calls `SpringApplication.run()`. Spring Boot creates the `ApplicationContext`, performs component scanning, applies auto-configuration, creates Beans, and starts the embedded web server.

Simple Example

```
@SpringBootApplication  
public class EmployeeApplication {
```

```
    public static void main(String[] args) {  
        SpringApplication.run(EmployeeApplication.class,  
args);  
    }  
}
```

Real Project Usage

Every Spring Boot application starts using `SpringApplication.run()`, making it the entry point for the application.

Interviewer Cross Question

What does `SpringApplication.run()` do?

Answer: It initializes the Spring Container, creates Beans, applies auto-configuration, and starts the embedded server.